Security Audit Report

MBR Login Customiser — WordPress Plugin

Audited version	**1.9.0**	Remediations	**1.9.1**
Report date	26 June 2026	Author	Robert Palmer
Vendor	Little Web Shack / Made by Robert	Distribution	Self-hosted (GPL)

Executive summary

MBR Login Customiser is a login-security plugin for WordPress that adds a custom login URL and appearance, attempt logging, IP whitelist/blacklist with CIDR support, brute-force lockouts, post-login redirect rules, time-based access windows, two-factor authentication (TOTP), and multisite network policy. This audit reviewed the full code base at version 1.9.0.

No critical or high-severity vulnerabilities were identified. The plugin consistently applies the expected defensive controls: parameterised database access, capability and nonce checks on every state-changing action, output escaping, timing-safe secret comparisons, an unforgeable client IP by default, a bounded lockout store, encrypted two-factor secrets, and direct-access guards on every file. Seven findings were raised, all in the low / informational / hardening band. The four code-level items have been resolved in version 1.9.1; the remainder are by-design behaviours and documentation notes.

Findings at a glance

ID	Finding	Severity	Status
F-01	2FA and Application Passwords interaction unhandled	Low-Med	Resolved 1.9.1
F-02	2FA secret encryption not authenticated (CBC)	Low	Resolved 1.9.1
F-03	Pending (pre-activation) 2FA secret unencrypted	Info	Resolved 1.9.1
F-04	Failed 2FA codes count toward IP lockout	Info	By design
F-05	Hidden login URL is obscurity, not a control	Info	Acknowledged
F-06	Logs store IP addresses and usernames (PII)	Info	Acknowledged
F-07	Mismatched internal docblock	Cosmetic	Resolved 1.9.1

Scope and methodology

The audit covered the complete plugin source at version 1.9.0: the bootstrap, the twelve per-concern modules under `includes/`, the bundled self-hosted updater, and the admin JavaScript. The review was a manual, white-box source read focused on the areas where login-security plugins typically fail, supported by an automated behavioural test harness.

- **Authentication and access control** — the authenticate filter chain, lockout and IP gating, the 2FA gate, and capability checks.
- **Injection and output safety** — database queries, custom CSS injected into the login page, and admin / login-form output.
- **Request integrity** — nonce / CSRF coverage on every action that changes state.
- **Cryptography** — TOTP correctness, secret storage, recovery-code handling, and comparison routines.
- **Client identity** — how the visitor IP is resolved, given the IP allow / deny lists and lockouts depend on it.
- **Multisite** — the network policy layer and new-site provisioning.

Behavioural verification was performed with a headless PHP harness that boots the plugin against WordPress stubs and executes every registered hook, asserting the security-relevant behaviours directly (120 assertions). TOTP was validated against the published RFC 4226 test vectors, and the two-factor secret encryption was checked by round-trip and tamper-rejection tests.

Independence note

Portions of the code under review were authored during the same engagement as this audit. The review was conducted critically and against the same bar regardless of authorship, but it is a structured self-review rather than a fully independent third-party assessment. The cryptographic and two-factor paths in particular would benefit from an additional independent expert review before being relied on in high-assurance contexts.

Controls verified as sound

- **SQL** — the log table writes through `$wpdb->insert` with an explicit format array, and every read uses `$wpdb->prepare`. Table names derive from the database prefix, never user input.
- **Client IP** — resolution defaults to `REMOTE_ADDR` (set by the web server, not forgeable by the client) and trusts a proxy header only when the administrator explicitly opts in and selects one. This closes the spoofing route that would otherwise let an attacker evade or weaponise IP-based lockouts.
- **Request integrity** — settings use the WordPress Settings API; the unlock and clear-log actions check `manage_options` plus a verified nonce; the network screen checks `manage_network_options` with a referer check; profile changes check `edit_user` and ride the core profile nonce.
- **Cryptographic comparisons** — TOTP and recovery-code checks use `hash_equals`. Secrets are generated with a 160-bit CSPRNG; recovery codes are single-use.
- **Output** — colours are validated with `sanitize_hex_color` at save; custom CSS is stripped of tags and style-element breakouts; footer HTML is filtered with `wp_kses_post`; the login field and profile output are escaped.
- **Resource limits** — the lockout store prunes expired entries and caps the number of tracked IPs, so it cannot be inflated without bound by logging in from many addresses.
- **Direct access** — every PHP file aborts unless `ABSPATH` is defined.

Detailed findings

F-01 2FA and Application Passwords

Low-Med | Resolved 1.9.1

Observation. The two-factor gate requires a code submitted with the interactive login form. WordPress also runs Application Password and REST / XML-RPC authentication through the same filter chain, where that code cannot be supplied, so for an enrolled user those requests previously failed closed (rejected). The behaviour was neither stated nor tested.

Risk. No bypass existed, but the silent fail-closed would break legitimate Application Password integrations for any two-factor user, and the policy was implicit.

Resolution. An explicit, documented policy was added on the Two-Factor tab. By default, Application Passwords are disabled for users who have two-factor enabled (via the application-password availability filter), removing any password-only path that skips the second factor. An ‘Allow’ mode is provided for sites that need those users to retain API access, in which a valid Application Password is treated as the API credential and exempted from the interactive code. The exemption keys off the core ‘did authenticate’ signal, which a client cannot forge, so an attacker cannot fake an app-password context to skip 2FA on the interactive form.

F-02 Authenticated encryption for 2FA secrets

Low | Resolved 1.9.1

Observation. The TOTP secret was encrypted at rest with AES-256-CBC, which provides confidentiality but not integrity (no authentication tag).

Risk. Low. The realistic threat is a database read (leak), which confidentiality already addresses, and no padding oracle is exposed. Tampering would require database write access. Still, unauthenticated encryption is below best practice for secret storage.

Resolution. Secret storage was moved to AES-256-GCM, an authenticated mode whose tag detects any modification of the stored ciphertext. Secrets written by earlier versions continue to decrypt and are upgraded to GCM when the user next re-enrols. Round-trip and tamper-rejection tests were added.

F-03 Pending two-factor secret

Info | Resolved 1.9.1

Observation. Before a user confirms enrolment, the candidate secret was stored in user meta in plain text.

Risk. Minimal — the pending value is not yet an active factor — but it is unnecessary exposure.

Resolution. The pending secret is now encrypted at rest with the same authenticated scheme as the active secret.

F-04 Failed codes and the lockout counter

Info | By design

Observation. A correct password followed by a wrong or missing two-factor code is recorded as a failed attempt, so repeated code errors can contribute to the submitting IP's lockout.

Risk. This is intentional — it rate-limits attempts to brute-force the second factor — but a legitimate user mistyping codes could lock out their own address.

Resolution. Retained as designed. Recommendation: keep the on-screen messaging clear, and consider a separate, more lenient counter for second-factor errors if support feedback warrants it.

F-05 Login URL hiding

Info | Acknowledged

Observation. Renaming or blocking the standard login endpoint reduces automated attack noise but is security through obscurity; alternative entry points (XML-RPC, REST, admin-ajax) remain.

Risk. Low. The feature is a useful noise filter, not a primary control, and could be over-trusted if framed as one.

Resolution. No code change required. Recommendation: document the feature honestly, positioning the lockout, IP gating and two-factor as the substantive controls.

F-06 Logged personal data

Info | Acknowledged

Observation. The attempt log stores IP addresses and usernames, which are personal data under regimes such as the UK GDPR.

Risk. Low / compliance. The Logs view is administrator-only, and retention is configurable with a default of 30 days.

Resolution. No code change required. Recommendation: keep the retention control and the privacy note visible so site owners can shorten retention or disable logging to meet their obligations.

F-07 Mismatched docblock

Cosmetic | Resolved 1.9.1

Observation. A documentation comment for the IP-resolution method had become detached from its function during an earlier edit.

Risk. None — cosmetic only.

Resolution. The comment was restored to its correct position.

Remediation summary (version 1.9.1)

All four code-level findings were fixed and verified before release. The changes are limited to the two-factor module, its admin tab, and one internal comment; no existing behaviour was removed.

- Application Passwords policy added (default: disabled for two-factor users), closing the password-only path and making the API behaviour explicit. **[F-01]**
- Two-factor secret storage upgraded to authenticated encryption (AES-256-GCM), with transparent reading of older secrets. **[F-02]**

– Pending enrolment secret encrypted at rest. **[F-03]**
– Internal comment corrected. **[F-07]**

Verification

The full automated suite passes at 120 assertions with zero failures, including the new tests for GCM round-trip, GCM tamper rejection, backward-compatible decryption of pre-existing secrets, encryption of the pending secret, and the Application Passwords policy in both modes. All fourteen PHP files pass lint.

Conclusion

At version 1.9.1, MBR Login Customiser presents a sound security posture for a plugin of its scope. The fundamentals that matter most for login security — unforgeable client identity, parameterised storage, request integrity, timing-safe secret handling, and now authenticated secret encryption with an explicit Application Passwords policy — are in place. No critical or high-severity issues were found, the code-level findings have been remediated and verified, and the residual items are by-design behaviours and documentation recommendations. As noted, an independent review of the cryptographic and two-factor paths is advisable before relying on the plugin in high-assurance environments.